

Lab 7: Database Connectivity

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

ระบบ Game Catalog ถูกออกแบบเป็นสถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) ประกอบด้วย Entity, Repository, Strategy, Service และ Controller โดยแต่ละคลาสถูกกำหนดหน้าที่ตามหลักการ GRASP

เพื่อให้ระบบมีความชัดเจนในการแบ่งความรับผิดชอบ ง่ายต่อการดูแลรักษาและขยายในอนาคต รายละเอียดการประยุกต์ใช้แต่ละ Pattern มีดังนี้

Controller Pattern

คลาส GameController ทำหน้าที่เป็นจุดรับ HTTP Request จาก Browser (GET/POST) เพียงอย่างเดียว โดยไม่มีการเขียน Business Logic หรือคำสั่งเข้าถึงฐานข้อมูลไว้ในคลาสนี้โดยตรง หน้าที่ของ Controller คือรับค่าพารามิเตอร์จาก Request แปลงเป็น Object แล้วส่งต่อไปให้ GameService ประมวลผล จากนั้นจึงเลือก View (Thymeleaf Template) ที่จะตอบกลับ หรือ redirect ไปยัง URL อื่น การจำกัดบทบาทของ Controller ให้แคบเช่นนี้ตรงตามหลัก Controller Pattern ของ GRASP ที่ระบุว่าควรมีคลาสหนึ่งทำหน้าที่รับผิดชอบการประสานงานระหว่าง UI และระบบภายใน โดยไม่ปะปนกับ Logic ทางธุรกิจ

High Cohesion

แต่ละคลาสในระบบมีหน้าที่เดียวที่ชัดเจนและสัมพันธ์กันภายในตัวเอง เช่น Game (Entity) มีหน้าที่เก็บโครงสร้างข้อมูลของเกมเท่านั้น, GameRepository มีหน้าที่เกี่ยวกับการเข้าถึงข้อมูลในฐานข้อมูลเท่านั้น, และ DiscountStrategy แต่ละคลาสมีหน้าที่คำนวณส่วนลดเพียงรูปแบบเดียว การที่แต่ละคลาสโฟกัสอยู่กับงานเดียวทำให้โค้ดอ่านง่าย ทดสอบง่าย และแก้ไขจุดใดจุดหนึ่งได้โดยไม่กระทบส่วนอื่น

Low Coupling

ความสัมพันธ์ระหว่างคลาสในระบบถูกออกแบบผ่าน Constructor Injection ของ Spring Framework เช่น GameController ไม่ได้สร้าง Object ของ GameService ขึ้นมาเอง (new GameService()) แต่รับเข้ามาผ่าน Constructor แทน ทำให้ Controller ไม่ผูกติดกับการ implement ภายในของ Service โดยตรง เช่นเดียวกับ GameService ที่พึ่งพา GameRepository และ DiscountContext ผ่าน interface/constructor เท่านั้น การลดการผูกติดกันโดยตรงเช่นนี้ทำให้สามารถเปลี่ยนแปลงการ implement ภายในของแต่ละ Layer ได้โดยไม่กระทบ Layer อื่น ตรงตามหลัก Low Coupling

Information Expert

หลักการนี้ระบุว่าควรมอบหมายความรับผิดชอบให้กับคลาสที่มีข้อมูลเพียงพอสำหรับทำหน้าที่นั้น ในระบบนี้ GameRepository เป็นผู้เชี่ยวชาญด้านการเข้าถึงข้อมูลเกมในฐานข้อมูล (เพราะ extends JpaRepository ที่มีข้อมูลการเชื่อมต่อ) จึงถูกมอบหมายให้ทำ CRUD กับฐานข้อมูลโดยตรง ในขณะที่ GameService เป็นผู้เชี่ยวชาญด้าน Business Logic เพราะมีทั้ง GameRepository และ DiscountContext อยู่ในมือ จึงถูกมอบหมายให้เป็นผู้ผูกข้อมูลเกม (Game) เข้ากับผลการคำนวณราคาส่วนลด (finalPrice, discountName) ก่อนส่งต่อไปยัง Controller

Indirection

คลาส DiscountContext ทำหน้าที่เป็นตัวกลาง (Intermediary) ระหว่าง GameService กับ Concrete Strategy ต่าง ๆ (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) โดย GameService ไม่จำเป็นต้องรู้ว่า มี Strategy ที่แบบ หรือแต่ละแบบคำนวณอย่างไร เพียงเรียกใช้ DiscountContext.calculatePrice(...) เท่านั้น

การมีตัวกลางเช่นนี้ช่วยลดการพึ่งพาโดยตรงระหว่าง Service กับ Strategy แต่ละคลาส สอดคล้องกับหลัก Indirection ของ GRASP ซึ่งช่วยเพิ่มความยืดหยุ่นให้ระบบ

1.2 High-Level SOLID Principles

ระบบออกแบบให้สอดคล้องกับหลักการ SOLID ทั้ง 5 ข้อ

หลักการ	การประยุกต์ใช้ในระบบ
S — Single Responsibility Principle	แต่ละคลาสมีหน้าที่รับผิดชอบเพียงหนึ่งเดียว: Controller จัดการ HTTP/View, Service จัดการ Business Logic, Strategy จัดการการคำนวณราคา, Repository จัดการการเข้าถึงฐานข้อมูล การเปลี่ยนแปลงในแต่ละส่วนจึงมีเหตุผลเดียวเสมอ (one reason to change)
O — Open/Closed Principle	หากต้องการเพิ่มส่วนลดรูปแบบใหม่ เช่น VIP Discount 30% สามารถสร้างคลาสใหม่ implement DiscountStrategy ได้ทันที โดยไม่ต้องแก้ไขโค้ดเดิมใน GameService หรือ DiscountContext (เปิดรับการขยาย แต่ปิดการแก้ไขโค้ดเดิม)
L — Liskov Substitution Principle	ทุกคลาสใน Strategy Package (NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy) implement interface DiscountStrategy และสามารถใช้แทนกันได้อย่างสมบูรณ์ผ่าน method calculateDiscountedPrice() โดยไม่ทำให้พฤติกรรมของระบบผิดเพี้ยนไปจากที่ควรจะเป็น

หลักการ	การประยุกต์ใช้ในระบบ
I — Interface Segregation Principle	interface DiscountStrategy มีเพียง method เดียวที่จำเป็น (calculateDiscountedPrice) ไม่มี method อื่นที่ไม่เกี่ยวข้องถูกบังคับให้ทุกคลาสต้อง implement ทำให้แต่ละ Strategy ไม่ต้องแบกรับ method ที่ไม่ใช้งาน
D — Dependency Inversion Principle	GameService ไม่พึ่งพา Concrete Class ของ Strategy โดยตรง แต่พึ่งพา Abstraction (DiscountStrategy interface ผ่าน DiscountContext) และรับเข้ามาผ่าน Constructor Injection เช่นเดียวกับ GameController ที่พึ่งพา GameService ผ่าน Constructor เท่านั้น ทำให้ Module ระดับสูงไม่ผูกติดกับรายละเอียดการ implement ของ Module ระดับล่าง

1.3 Strategy Pattern

ระบบใช้ Strategy Pattern ในการคำนวณราคาสุทธิของเกมตามประเภทส่วนลดที่ผู้ใช้เลือก แทนที่จะเขียนเงื่อนไข if-else หรือ switch-case คำนวณราคาไว้ในที่เดียว (เช่นใน Service หรือ Controller)

ซึ่งจะทำให้ได้ขยายยากเมื่อมีส่วนลดรูปแบบใหม่เพิ่มเข้ามา โครงสร้าง Strategy Pattern ที่ใช้มีองค์ประกอบดังนี้

- DiscountStrategy (Interface) — กำหนดสัญญากลางว่าทุก Strategy ต้องมี method calculateDiscountedPrice(double originalPrice) ที่คืนค่าราคาหลังหักส่วนลด
- NoDiscountStrategy — คืนราคาเดิมโดยไม่มีการหักส่วนลด (0%) ใช้เมื่อ discountType = NONE
- StudentDiscountStrategy — คำนวณราคาหลังหักส่วนลด 10% ($\text{originalPrice} \times 0.90$) ใช้เมื่อ discountType = STUDENT
- SeasonalSaleStrategy — คำนวณราคาหลังหักส่วนลด 20% ($\text{originalPrice} \times 0.80$) ใช้เมื่อ discountType = SEASONAL
- DiscountContext — เป็นตัวกลาง (Context) ที่รับค่า discountType เข้ามา แล้วเลือก Concrete Strategy ที่เหมาะสมให้โดยอัตโนมัติผ่าน switch expression จากนั้นเรียก method คำนวณของ Strategy ที่เลือกและคืนผลลัพธ์กลับไป

ประโยชน์ที่ชัดเจนที่สุดของการใช้ Strategy Pattern ในกรณีนี้คือการสนับสนุนหลัก Open/Closed Principle (OCP) กล่าวคือ

หากในอนาคตต้องการเพิ่มโปรโมชั่นใหม่ เช่น ส่วนลดสมาชิก VIP 30% หรือส่วนลดช่วงเทศกาลพิเศษ 50%

ผู้พัฒนาเพียงสร้างคลาสใหม่ที่ implement DiscountStrategy และเพิ่ม case ใน DiscountContext เท่านั้น

โดยไม่ต้องแก้ไขโค้ดเดิมของ GameService, GameController หรือ Strategy อื่น ๆ ที่มีอยู่แล้วเลย

ซึ่งช่วยลดความเสี่ยงที่การแก้ไขจุดหนึ่งจะไปกระทบส่วนที่ทำงานถูกต้องอยู่แล้ว (regression)

และทำให้ระบบสามารถขยายความสามารถได้อย่างปลอดภัยในระยะยาว

1.4 Layered Architecture

ระบบแบ่งออกเป็น 4 ชั้นหลัก ได้แก่ Controller Layer, Service Layer, Strategy Layer และ Repository/Data Access Layer โดยแต่ละคำขอจาก Browser จะไหลผ่านชั้นเหล่านี้ตามลำดับ เหตุผลสำคัญที่ต้องแยก Service Layer ออกจาก Controller และ Repository อย่างชัดเจนมีดังนี้

- **Low Coupling:** หาก Controller เรียก GameRepository ตรง ๆ โดยไม่ผ่าน Service, Controller จะผูกติดกับรายละเอียดการเข้าถึงข้อมูลและ Business Logic ไปพร้อมกัน เมื่อกฎทางธุรกิจเปลี่ยน (เช่น เปลี่ยนวิธีคำนวณส่วนลด) จะต้องแก้ไขได้ใน Controller โดยตรง การมี Service คั่นกลางทำให้ Controller และ Repository ไม่ต้องรู้จักกันโดยตรง แต่ละฝั่งเปลี่ยนแปลงได้อย่างอิสระ
- **High Cohesion:** Service Layer รวบรวม Business Logic ทั้งหมดไว้ในที่เดียว (เช่น การผูก Game เข้ากับผลการคำนวณราคาผ่าน DiscountContext) ทำให้ Controller คงความรับผิดชอบเดียวคือจัดการ HTTP Request/Response และ Repository คงความรับผิดชอบเดียวคือการเข้าถึงฐานข้อมูล แต่ละ Layer จึงมีความเชื่อมโยงภายในสูงและไม่ปะปนหน้าที่กัน

นอกจากนี้การแยก Layer ยังช่วยให้ทดสอบระบบได้ง่ายขึ้น (สามารถทดสอบ Service แยกจาก Controller หรือ Database ได้) และรองรับการเปลี่ยนแปลงเทคโนโลยีในอนาคต เช่น หากต้องการเปลี่ยนจาก PostgreSQL เป็นฐานข้อมูลอื่น จะกระทบเฉพาะ Repository Layer เท่านั้น โดยที่ Service และ Controller ไม่ต้องแก้ไข

1.5 Execution Flow

ลำดับการทำงานของระบบเมื่อผู้ใช้เพิ่มเกมใหม่ผ่านฟอร์ม (ตัวอย่าง Flow ที่ครอบคลุมการทำงานของทุก Layer) เป็นดังนี้

ลำดับ	รายละเอียดการทำงาน
1	Browser ส่ง HTTP GET ไปที่ /games/add เพื่อขอฟอร์มเพิ่มเกม
2	GameController รับ Request ผ่าน @GetMapping("/add") สร้าง Object Game วางเปล่า ใส่ใน Model แล้วคืน View "games/add" (add.html) กลับไปแสดงผล
3	ผู้ใช้กรอกข้อมูลเกม (title, genre, platform, rating, price, discountType, releaseDate) แล้วกด บันทึก ซึ่ง Browser จะส่ง HTTP POST ไปที่ /games/save พร้อมข้อมูลฟอร์ม
4	GameController รับ Request ผ่าน @PostMapping("/save") ผูกข้อมูลจากฟอร์มเข้ากับ Object Game โดยอัตโนมัติ (@ModelAttribute) แล้วเรียก gameService.saveGame(game)
5	GameService รับ Object Game และส่งต่อให้ gameRepository.save(game) เพื่อบันทึกข้อมูล
6	GameRepository (Spring Data JPA) แปลง Object เป็นคำสั่ง SQL INSERT ผ่าน Hibernate แล้วส่งไปบันทึกจริงที่ฐานข้อมูล PostgreSQL (lab7demo)
7	หลังบันทึกสำเร็จ Controller ตั้งค่าข้อความแจ้งเตือนผ่าน RedirectAttributes (flash attribute) แล้ว redirect กลับไปที่ /games
8	Browser ส่ง HTTP GET ใหม่ไปที่ /games โดยอัตโนมัติจาก redirect
9	GameController เรียก gameService.getAllGames() เพื่อดึงข้อมูลเกมทั้งหมด
10	GameService เรียก gameRepository.findAll() ดึงข้อมูลเกมทั้งหมดจากฐานข้อมูล จากนั้นวนลูปแต่ละเกมเพื่อคำนวณราคาสุทธิ โดยเรียก discountContext.calculatePrice(discountType, price) ซึ่ง DiscountContext จะเลือก Concrete Strategy ที่ตรงกับ discountType ของเกมนั้น แล้วคำนวณราคาคืนกลับมา
11	ผลลัพธ์ (finalPrice, discountName) ถูกผูกเข้ากับแต่ละ Object Game แล้วส่งกลับไปยัง Controller
12	GameController ใส่รายการเกมทั้งหมดใน Model แล้วคืน View "games/list" (list.html) ซึ่ง Thymeleaf จะ render ตารางแสดงรายการเกม พร้อมข้อความแจ้งเตือนสีเขียวและราคาสุทธิที่คำนวณผ่าน Strategy Pattern ให้ผู้ใช้เห็นบน Browser

ส่วนที่ 2: Code Implementation & Explanation

ในทุก Layer ของระบบใช้ Dependency Injection รูปแบบ Constructor Injection ของ Spring Framework เพียงรูปแบบเดียว (ไม่ใช่ @Autowired บน Field โดยตรง) เนื่องจาก Constructor Injection ทำให้ Dependency ของแต่ละคลาสถูกประกาศชัดเจนตั้งแต่จุดสร้าง Object, บังคับให้ Dependency เป็น final (immutable) ป้องกันการเปลี่ยนแปลงโดยไม่ตั้งใจ, และทำให้เขียน Unit Test ได้ง่าย เพราะสามารถส่ง Mock Object เข้าไปทาง Constructor ได้โดยตรงโดยไม่ต้องพึ่งพา Spring Container

2.1 Entity — Game.java

เป็นคลาสที่ map กับตาราง games ในฐานข้อมูลผ่าน JPA Annotation (@Entity, @Id, @GeneratedValue) มีหน้าที่เก็บโครงสร้างข้อมูลของเกมเพียงอย่างเดียว ประกอบด้วย field ตามที่ Template ต้องการ ได้แก่ id, title, genre, platform, rating, releaseDate, price และ discountType นอกจากนี้ยังมี field แบบ @Transient สองตัวคือ finalPrice และ discountName ซึ่งเป็นค่าที่คำนวณได้จาก Strategy Pattern ในระหว่างการทำงาน จึงไม่ถูกบันทึกลงฐานข้อมูลจริง แต่ใช้สำหรับส่งค่าไปแสดงผลที่หน้า list.html เท่านั้น

```
@Entity
@Table(name = "games")
public class Game {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String genre;
    private String platform;
    private Double rating;
    private LocalDate releaseDate;
    private Double price;
    private String discountType; // NONE, STUDENT, SEASONAL

    // ค่าที่คำนวณได้ ไม่บันทึกลง DB
    @Transient
```

```
private Double finalPrice;  
  
@Transient  
private String discountName;
```

2.2 Repository — GameRepository.java

เป็น Data Access Layer ที่ทำหน้าที่เข้าถึงฐานข้อมูลโดยตรง โดย extends JpaRepository<Game, Long> ของ Spring Data JPA ซึ่งมี method พื้นฐานสำหรับ CRUD ให้ใช้งานทันที (findAll, findById, save, deleteById) โดยไม่ต้องเขียนคำสั่ง SQL เอง เป็นไปตามหลัก Information Expert เพราะคลาสนี้เป็นผู้เชี่ยวชาญด้านการเชื่อมต่อฐานข้อมูลโดยตรง

```
@Repository  
public interface GameRepository extends JpaRepository<Game, Long> {  
}
```

2.3 Strategy Package

ประกอบด้วย Interface หนึ่งตัวและ Concrete Strategy สามตัว บวกกับ Context หนึ่งตัวที่ทำหน้าที่เลือก Strategy ให้อัตโนมัติ ทุกคลาสถูก annotate ด้วย @Component เพื่อให้ Spring จัดการ Object เหล่านี้ใน Container และสามารถ Inject เข้าไปใน DiscountContext ผ่าน Constructor ได้

```
public interface DiscountStrategy {
    double calculateDiscountedPrice(double originalPrice);
}

@Component
public class StudentDiscountStrategy implements DiscountStrategy {
    public double calculateDiscountedPrice(double originalPrice) {
        return originalPrice * 0.90; // ลด 10%
    }
}

@Component
public class DiscountContext {
    private final NoDiscountStrategy noDiscountStrategy;
    private final StudentDiscountStrategy studentDiscountStrategy;
    private final SeasonalSaleStrategy seasonalSaleStrategy;

    public DiscountContext(NoDiscountStrategy n, StudentDiscountStrategy s,
                          SeasonalSaleStrategy se) {
        this.noDiscountStrategy = n;
        this.studentDiscountStrategy = s;
        this.seasonalSaleStrategy = se;
    }

    public double calculatePrice(String discountType, double price) {
        DiscountStrategy strategy = switch (discountType) {
            case "STUDENT" -> studentDiscountStrategy;
            case "SEASONAL" -> seasonalSaleStrategy;
            default -> noDiscountStrategy;
        };
        return strategy.calculateDiscountedPrice(price);
    }
}
```

ในตัวอย่างนี้ DiscountContext รับทั้งสาม Strategy เข้ามาผ่าน Constructor Injection พร้อมกัน (ไม่ได้ inject ผ่าน List<DiscountStrategy> เพื่อความชัดเจนในระดับ Lab) ซึ่งแสดงให้เห็นการฉีด Dependency หลายตัวพร้อมกันในคลาสเดียว และคงสอดคล้องกับหลัก Dependency Inversion เพราะ GameService ที่เรียกใช้งานต่อ ก็รับ DiscountContext เข้ามาผ่าน Constructor เช่นกัน โดยไม่ต้องรู้จัก Concrete Strategy ทั้งสามตัวโดยตรง

2.4 Service — GameService.java

เป็น Business Logic Layer ที่รับทั้ง GameRepository และ DiscountContext เข้ามาผ่าน Constructor Injection มีหน้าที่ประสานงานระหว่างการดึงข้อมูลจากฐานข้อมูลกับการคำนวณราคาส่งออก โดยก่อนส่งข้อมูลเกมกลับไปให้ Controller จะเรียก DiscountContext เพื่อคำนวณ finalPrice และ discountName แล้วผูกเข้ากับแต่ละ Object Game ก่อนเสมอ

```
@Service
public class GameService {
    private final GameRepository gameRepository;
    private final DiscountContext discountContext;

    public GameService(GameRepository gameRepository,
                       DiscountContext discountContext) {
        this.gameRepository = gameRepository;
        this.discountContext = discountContext;
    }

    public List<Game> getAllGames() {
        List<Game> games = gameRepository.findAll();
        games.forEach(this::applyDiscountInfo);
        return games;
    }

    private void applyDiscountInfo(Game game) {
        double finalPrice = discountContext.calculatePrice(game.getDiscountType(), game.getPrice());
        game.setFinalPrice(finalPrice);
        game.setDiscountName(discountContext.getDiscountName(game.getDiscountType()));
    }
}
```

2.5 Controller — GameController.java

เป็นจุดรับ HTTP Request ทั้งหมดของระบบ ครอบคลุมการทำงาน CRUD ครบทั้ง 4 ฟังก์ชันตาม URL Mapping ที่ Template ต้องการ (list, add/save, edit/update, delete) โดยรับ GameService เข้ามาผ่าน Constructor Injection เพียงตัวเดียว และไม่มีเรียก GameRepository โดยตรงเลย เพื่อรักษาหลัก Low Coupling ระหว่าง Controller กับ Data Access Layer

```
@Controller
@RequestMapping("/games")
public class GameController {
    private final GameService gameService;

    public GameController(GameService gameService) {
        this.gameService = gameService;
    }

    @GetMapping
    public String listGames(Model model) {
        model.addAttribute("games", gameService.getAllGames());
        return "games/list";
    }

    @PostMapping("/save")
    public String saveGame(@ModelAttribute Game game,
                           RedirectAttributes redirectAttributes) {
        gameService.saveGame(game);
        redirectAttributes.addFlashAttribute("message", "เพิ่มเกมสำเร็จ!");
        return "redirect:/games";
    }
}
```

จุดที่ควรสังเกตคือทุกเมธอดที่ทำการ redirect กลับไปหน้า /games หลังบันทึก/อัปเดต/ลบข้อมูล จะใช้

RedirectAttributes.addFlashAttribute(...) แทนการใช้ Model.addAttribute(...) ธรรมดา เนื่องจาก redirect เป็นการสร้าง HTTP Request ใหม่อีกรอบ ค่าที่เก็บไว้ใน Model ปกติจะไม่ถูกส่งต่อข้าม Request แต่ Flash Attribute

ถูกออกแบบมาสำหรับส่งข้อมูลชั่วคราวข้าม redirect โดยเฉพาะ ทำให้ข้อความแจ้งเตือน (message) ปรากฏใน list.html ได้ถูกต้องหลัง redirect

ส่วนที่ 3: Web Application & Database Screenshots

ภาพหน้าจอทั้งหมดด้านล่างแสดงการทดสอบระบบจริง โดยข้อมูลเกมที่เพิ่มเข้าไประบุรหัสนักศึกษาและ Section กำกับไว้ในช่อง Title ตามข้อกำหนดของ Lab

3.1 หน้าจอการเพิ่มเกมใหม่ (Create)

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

+

เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม

Monster Hunter Wilds (673380392-1 sec 3)

แนวเกม (Genre)

Action

แพลตฟอร์ม (Platform)

PC

☆ คะแนน (Rating)

10

📏 ราคาปกติ (บาท)

1090

% ส่วนลด (Strategy)

ส่วนลดนักศึกษา (10%)

ราคาปกติ (0%)

ส่วนลดนักศึกษา (10%)

ส่วนลดเทศกาล (20%)

📅 วันวางจำหน่าย (Release Date)

02 / 28 / 2025

บันทึก

ยกเลิก

Game Catalog

CP353002 — Lab 7

← กลับไปรายการเกม

+

เพิ่มเกมใหม่

กรอกข้อมูลเกมที่ต้องการเพิ่มลงในระบบ

ชื่อเกม

Baldur's Gate 3 (673380392-1 sec 3)

แนวเกม (Genre)

RPG

แพลตฟอร์ม (Platform)

PC

☆ คะแนน (Rating)

10

📏 ราคาปกติ (บาท)

1790

% ส่วนลด (Strategy)

ส่วนลดเทศกาล (20%)

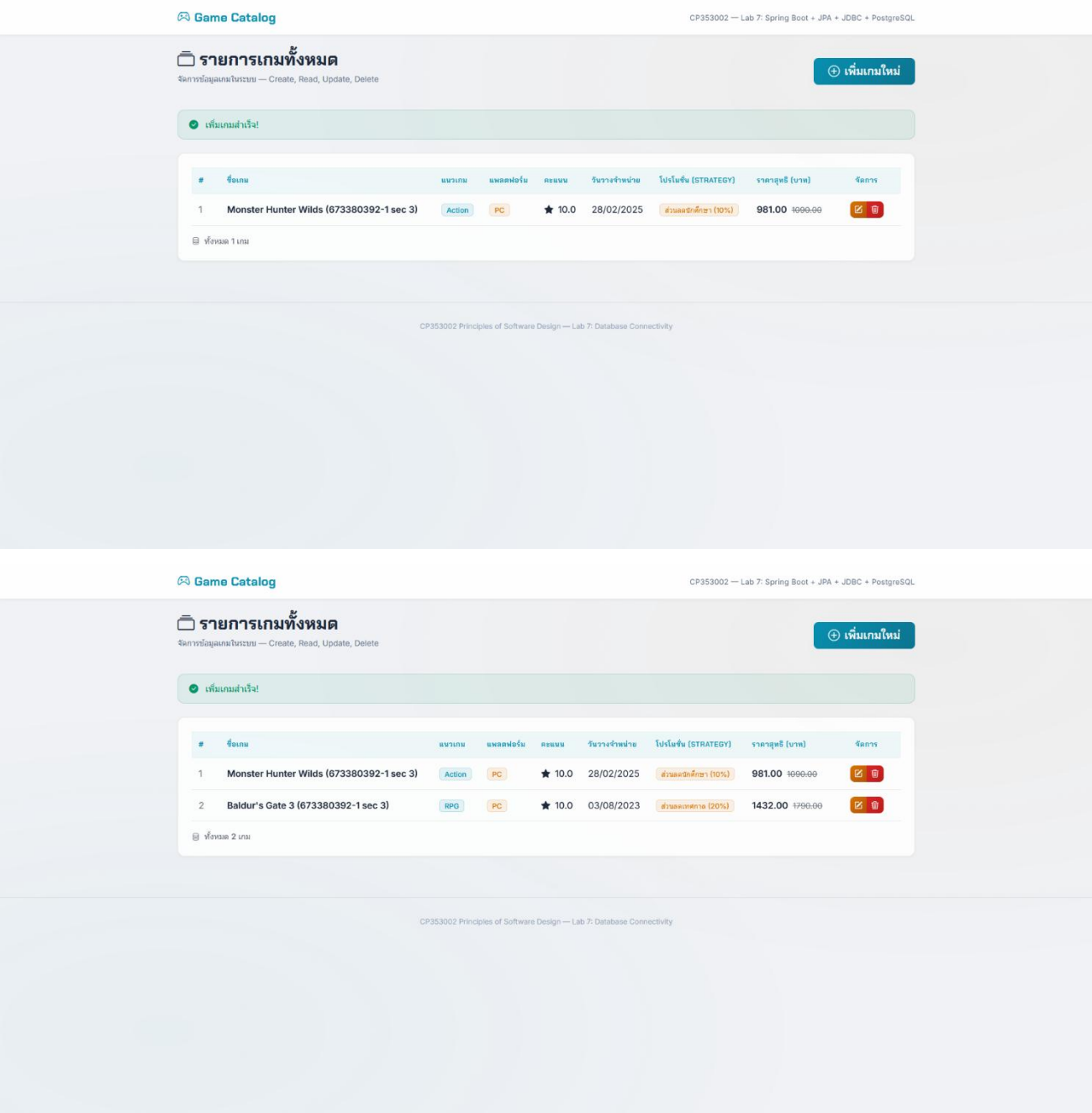
📅 วันวางจำหน่าย (Release Date)

08 / 03 / 2023

บันทึก

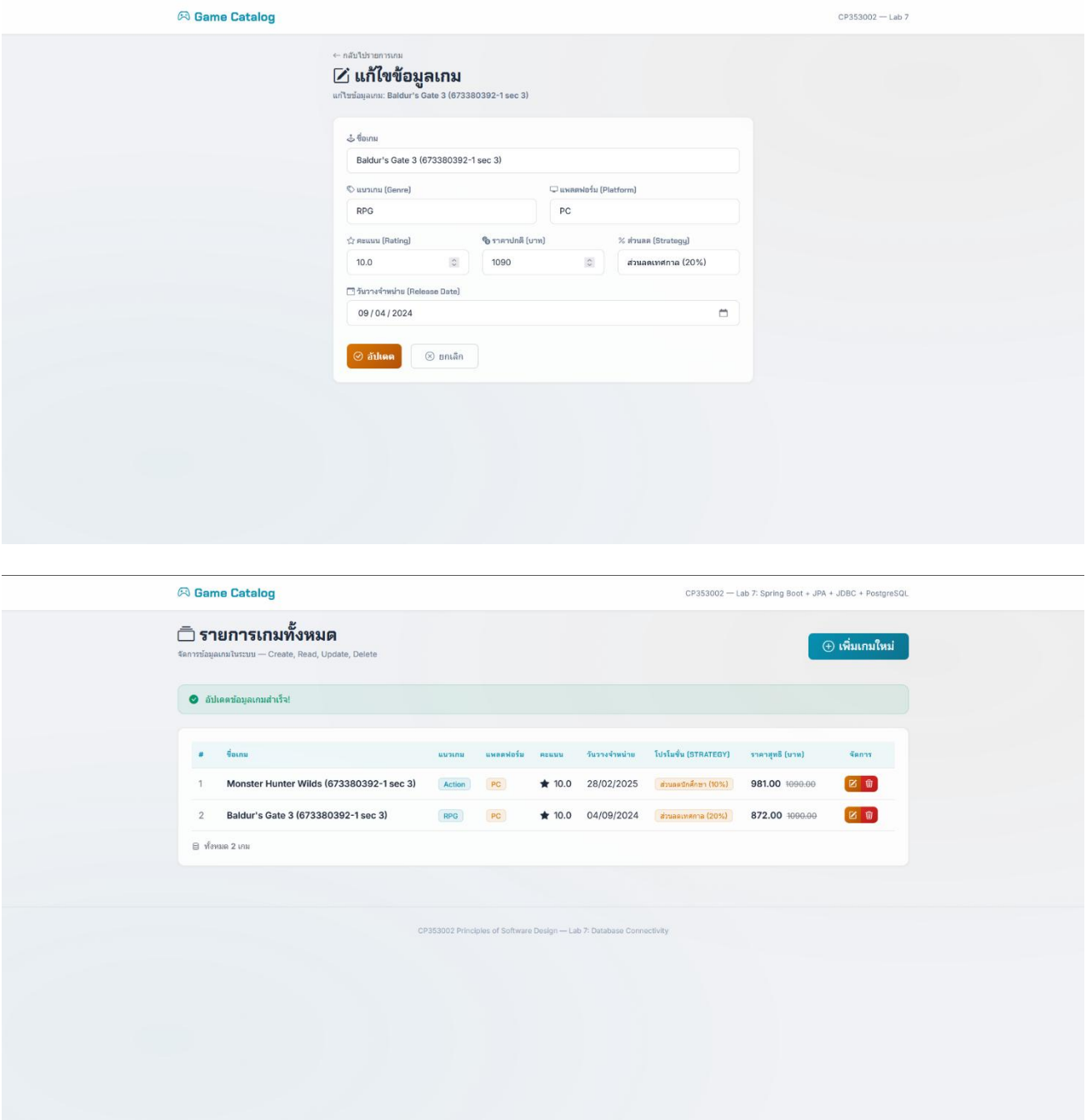
ยกเลิก

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)

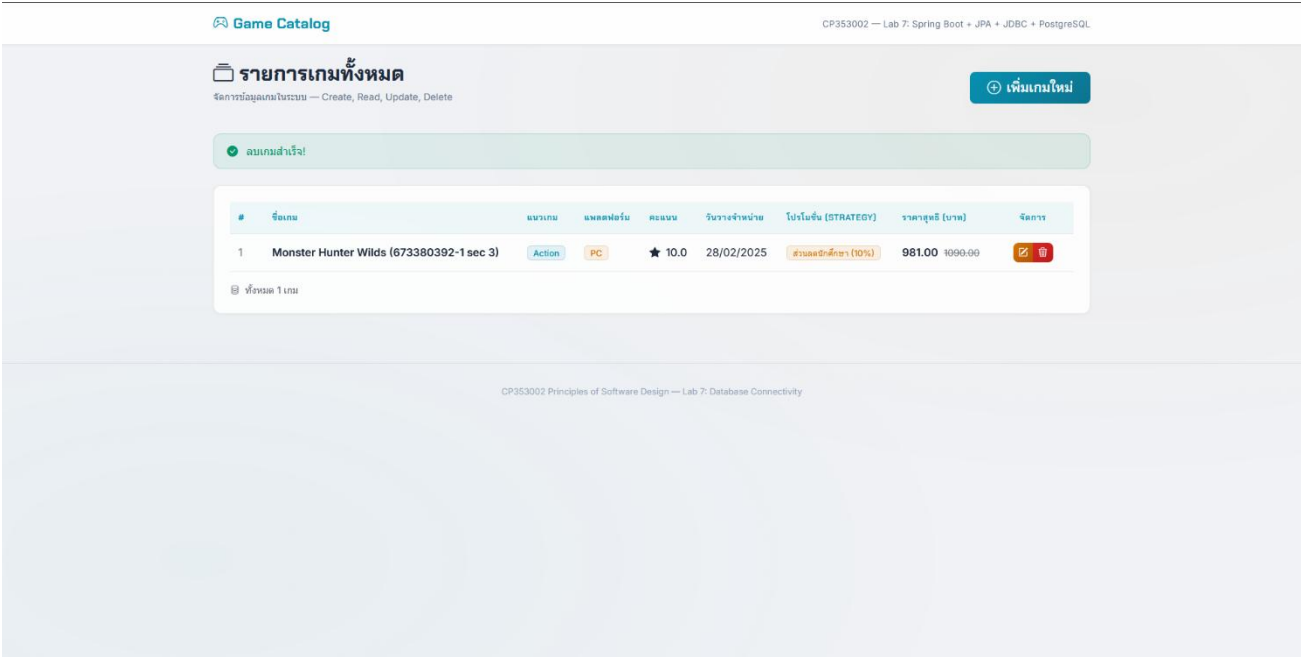
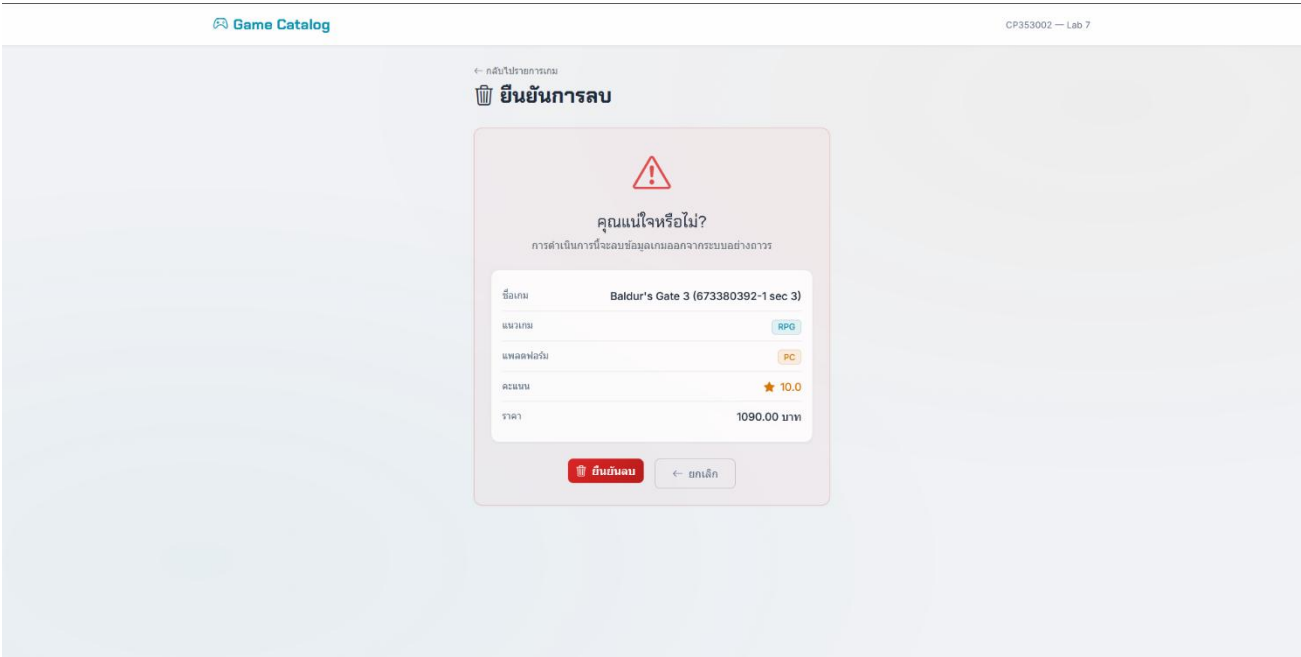


รูปที่ 3.2: หน้าแสดงรายการเกมทั้งหมด พร้อมราคาสุทธิที่คำนวณผ่าน Strategy Pattern

3.3 หน้าจอแก้ไขข้อมูลเกม (Update)



3.5 หน้าจอยืนยันและการลบข้อมูลเกม (Delete)



3.7 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database

The screenshot displays a PostgreSQL database management interface. At the top, there are tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, and Processes. The main window shows a query editor with the text `SELECT * FROM games;`. Below the editor, the 'Data Output' tab is active, showing a single row of data. The row contains the following values: 1, 2, STUDENT, Action, PC, 1090, 10, 2025-02-28, and Monster Hunter Wilds (673380392-1 sec 3). The interface also includes a 'Scratch Pad' tab and a 'Messages' section.

id	discount_type	genre	platform	price	rating	release_date	title	
1	2	STUDENT	Action	PC	1090	10	2025-02-28	Monster Hunter Wilds (673380392-1 sec 3)